Bulk Certificate Generator

Updated Automated Testing Guide

Purpose and test environment

This guide gives copyable PowerShell commands and expected results for the current test suite. Tests call the FastAPI app through TestClient and use isolated in-memory SQLite plus temporary file storage. Do not start Uvicorn for pytest.

Setup

```
cd C:\path\to\bulk-certificate-generator
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

If activation is blocked: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass, then run .\.venv\Scripts\Activate.ps1.

Run all tests with coverage

```
pytest --cov=app
```

Expected: 10 passed. A Starlette/httpx deprecation warning may be displayed depending on dependency versions; it is not a test failure. Latest observed app coverage was 94%.

Run a single required test

```
python -m pytest -q tests/test_enhancements.py::test_csv_upload_job_creation
python -m pytest -q tests/test_enhancements.py::test_rate_limiting_trigger
python -m pytest -q tests/test_enhancements.py::test_concurrency_guard_execution
```

Test Cases and Expected Results

Coverage area	Input / execution	Expected output
Generation job	POST JSON with event Python Workshop and 3 fictional recipients	202 Accepted; PENDING response; later COMPLETED, 3 successful
Input validation	Empty recipients, malformed email, empty name	Each request returns 422
Certificate generation	Render Test Event certificate for Taylor Example, grade A	Bytes start with %PDF- and exceed 500 bytes
Status and progress	Poll the 3-recipient completed job	COMPLETED, 3/3 done, progress 100%
Individual render failure	Renderer raises for Grace Hopper in a 3-person batch	2 successes, 1 failure, PARTIALLY_COMPLETED; later recipients still finish
Certificate retrieval	Download successful recipient and job archive	PDF response is application/pdf; bulk is application/zip
CSV ingestion	Multipart CSV name,email,department,grade with two valid rows	202 Accepted; 2 recipient rows stored; extra columns in metadata
Rate limit	Send 11 rapid JSON job POSTs from one client IP	First 10 return 202; request 11 returns 429
Concurrency guard	Run 6 recipient tasks with MAX_WORKERS patched to 2	Job completes; observed peak generator concurrency is 2 and never exceeds the configured bound

The three new enhancement tests live in tests/test_enhancements.py. Existing job and certificate tests cover the other cases.

Manual Scenario and Benchmark Runs

Manual batch scenario command

```
python -m tests.manual_scenarios
```

Runs fictional batches of 1, 5 (one intentionally injected rendering failure), 100, and 1,000 recipients. It writes ZIP bundles under storage\certificates\. The 1,000-recipient run is an observed test size, not a configured maximum.

HTTP benchmark

The benchmark requires a running local API. In PowerShell window 1 start the service; in window 2 run the script:

```
# Window 1
python -m uvicorn app.main:app --reload

# Window 2
python scripts/benchmark.py
```

The script submits 100 recipients, polls until a terminal state, and prints job ID, success/failure counts, elapsed seconds, and successful certificates per second. Options: --base-url, --poll-interval, --timeout.

Latest verified benchmark: 100/100 completed, 1.74 seconds, 57.38 certificates/second. Host load and polling intervals affect measurements; use them as local observations only.

Troubleshooting

If pytest is not found, activate .venv and install requirements.txt. If multipart requests fail because a package is missing, reinstall requirements. If rate-limit assertions fail due to test order, run the entire suite: the rate-limit test resets its in-memory limiter state before and after its assertions. For benchmark connection errors, confirm Uvicorn is running at the base URL supplied.

No server process is needed for the automated suite. The benchmark is the only step here that calls a separately running API process.